Assignment 3

6 Points Possible

11/2/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

11/2/2026

Details

Assignment 3: Heap Go Player with Simulation Policies

Due: **Monday, Nov 2**, 23:55. Submit here on Canvas.

Responsible TA for this assignment: Shuwei (see teaching team page)

Resources

- Starter code: `assignment3.tgz` **(https://canvas.ualberta.ca/courses/34644/files/9416017?wrap=1)**
  - Decompress with `tar -xvzf assignment3.tgz`
  - The starter code contains an implementation of Heap Go and related text commands (as in Assignment 2 starter code), plus some commands for you to implement or modify for this Assignment 3.
- Review these as needed:
  - **All assignments - Rules and Policies (https://canvas.ualberta.ca/courses/34644/assignments/776003)**
  - **Rules of Heap Go (https://canvas.ualberta.ca/courses/34644/pages/rules-of-heap-go)** , **Assignment 1 (https://canvas.ualberta.ca/courses/34644/assignments/813356)** and **Assignment 2 (https://canvas.ualberta.ca/courses/34644/assignments/809162)**
  - **SSH connection guide (https://canvas.ualberta.ca/courses/34644/pages/ssh-connection-guide)** - connect to UCOMM machines (required for testing your code)
  - **Lecture slides (https://canvas.ualberta.ca/courses/34644/pages/lecture-slides)** and **sample code (https://github.com/ualberta-mueller-group/CMPUT455_code)**
  - **Measuring Time in Python (https://canvas.ualberta.ca/courses/34644/pages/measuring-time)**
- **Assignment 3 preview slides (https://canvas.ualberta.ca/courses/34644/files/9452287?wrap=1)** **(https://canvas.ualberta.ca/courses/34644/files/9452287/download?download_frd=1)** from class

In this assignment, you develop two different types of playout policies for Heap Go, and a simple simulation-based player that uses them. You add your code to our starter code `a3.py`. This includes a random Heap Go player similar to the starter code of Assignment 2.

Assignment 3 Specification

Write a Python3 program for randomised simulations of Heap Go positions. Your solution is based on our `a3.py` starter code from `assignment3.tgz`.

Heap Go Restrictions for this Assignment

Same as in Assignment 2. At the start of each game:

- The number of heaps is in the range **1 ≤ number of heaps ≤ 20**
- Each heap contains a number of tokens, in the range **1 ≤ tokens in heap ≤ 20**
- Each token (c, n) has color c, either 'b' or 'w', and integer value n, in the range 1 ≤ n ≤ 20
- The komi is an integer + 0.5 to avoid draws, in the range -100 < komi < +100

Other Restrictions for this Assignment

For this assignment, we will only test your `a3.py` program with valid input. Error handling will not be tested. As always, a successful command, even if it has no other output, should print the command status at the end:

```
= 1
```

There should not be any failed commands with status -1 in this assignment, but leaving in the existing error handling could still be useful, e.g. to catch typos.

The **resource limits** for running and submitting your code are as in Assignment 2: one thread, 1 GB of memory at runtime, max. 1MB submission file size.

We will attempt to reply to all student questions, but you are expected to read about the assignment first. If the answer is already in the assignment guidelines, or in this spec, or linked from this spec, or in the discussion, then our reply might be short, as in "read the spec".

Getting Started

Download `assignment3.tgz` and expand it into a directory `assignment3`, which contains:

- Our starter code for assignment 3, `a3.py`. This implements a random player, and contains placeholders for commands that you need to implement/modify.
- A file `assignment3-public-tests.txt`, with test cases for your program. Also use this for your presubmission test. See details below.
- A program `a3test.py`. Use this code to test your solution with

  ```
  python3 a3test.py a3.py assignment3-public-tests.txt
  ```

Goals of the Assignment

Implement the following functionality by adding or modifying existing text commands. Examples of each command and output are in the public tests.

Part 1: Pure Random Simulations

Implement a command

```
simulate
```

This command runs one simulation and outputs the winner of that simulation, `b` or `w`.

In Part 1, a simulation consists of a series of legal moves generated uniformly at random from a given start state. (In part 3 we will add another type of simulation.) Each simulation ends when the game is over, and returns the winner of the game.

The details of how you implement this function are up to you. However, you need to make sure that at the end, the game state is reset to exactly what it was before the `simulate` command. For example, when calling this command several times in a row it should always start from the same, current Heap Go state.

Example: Assume that you have a Heap Go position with 7 heaps, and heaps 0 and 4 are currently empty. That leaves 5 legal moves - on heaps 1, 2, 3, 5 and 6. Your simulation should randomly pick and play one of these moves, with a probability of 1/5 each. Then it should continue playing out random moves until the game is over. In the end, check which color wins, using the final score including the komi.

Part 2: Simulation-based Flat Monte Carlo Heap Go Player

Implement a command

```
numsimulations n
```

Where n is an integer with $1 \leq n \leq 100$. This represents the number of simulations to use **per move**. The default setting, used before the first `numsimulations` command, is to use n=10 simulations per move.

Change the `genmove` command in `a3.py`: To generate a computer move in current state s: For each legal move m from s: play m, then evaluate it by running a fixed number n of simulations, then undo m. Afterwards, pick one move to play - a move with the highest win percentage for toplay in s. You can break ties in any way you like.

The number of simulations n is given by the most recent `numsimulations` command (or the default if there was no such command yet - see the command spec).

Example: see the sample code `simulation_player.py` in the repo for a sample implementation of

the algorithm. You are free to break ties between moves with equal winrate in any way you wish.

`genmove` will not be called when the game is over, so you do not need error handling for such a case.

Part 3: Greedy-based Probabilistic Simulations for your Player

In probabilistic simulations, different legal moves may have different probabilities of being chosen in the simulation. The probabilistic simulation policy in this part of the assignment changes the way simulations are done. Instead of choosing each legal move with equal probability, it more often chooses moves that take more points immediately.

We define the heuristic **weight w** of a move as the total value of all tokens that are taken by this move. For example, if a move takes a single token of value 7, then its weight is w=7. As another example, a move that takes three tokens of values 5, 6 and 2 has weight w = 5 + 6 + 2 = 13.

From Weights to Probabilities

In a probabilistic policy, each move is selected in proportion to its weight. To compute the move probabilities, compute the sum W of all move weights, and divide the weight of each move by W.

Example: given three legal moves with weights 5, 12 and 3. Then W = 5+12+3 = 20, and the probabilities of choosing these moves are 5/20 = 0.25, 12/20 = 0.6 and 3/20 = 0.15 accordingly.

Commands

Implement a command

```
policy type
```

Set the playout policy to be used in your player from now on. The argument `type` is either `random` or `probabilistic`. When your program first starts up, before the first such command, the policy type should be `random`.

Make sure that your `simulate` and `genmove` commands also respect the current setting of policy type.

Part 4: Show the Probabilities for the next move for your Player

Implement a command

```
policymoves
```

This command should print a sorted list of all legal moves and their probabilities, rounded to exactly three digits after the decimal point, for the current simulation policy in the format

```
move1 prob1 move2 prob2 ...movenprobn
```

```
= 1
```

Moves should be numerically sorted in increasing order by heap number. For example, if the move on heap 0 has 20% probability and move 1 has 80%, then output

```
0 0.200 1 0.800
```

As another example, if the current policy type is random and you have 4 legal moves on heaps 2, 3, 5 and 6, then output

```
2 0.250 3 0.250 5 0.250 6 0.250
```

Do not include illegal moves in any of your output. If the command is called at the end of game, this is not an error but there should simply be no output, only the status line afterwards.

Other Commands

The functionality of other existing text commands in the starter code should be preserved. Especially, `heapgo, play, toplay` are used to set up test cases.

Pre-submission Test and Submission

Follow the same general steps as in Assignments 1 and 2 to create your submission, test it on a lab machine, and create a log file. Use your `assignment3` directory, `assignment3.tgz` as submission file name, and test with `assignment3-public-tests.txt`. Remember to include your current log file and `readme.txt` in your submission. The AI use form is recommended but optional.

Also, similar to previous Assignments:

- The submission must be a valid tgz file named `assignment3.tgz`
- Leave the name and location of your `a3.py` main file as in the starter code.
- Do not add directories or change the `assignment3` directory name.
- You may add extra Python files within the same directory.

Hints and Details - Read them All

- The Python function `random.choices` **(https://docs.python.org/3/library/random.html#random.choices)** can deal with both uniform random and weight-based probabilistic selection.
- In the output text only, not for computation: Make sure to round the printed probabilities to 3 digits of precision following standard rounding rules.
- Time management: the test script is set to a time limit of 1 second per command. Our simple sample solution stays far below that limit even on large games.
- Set up your Assignment 3 Group, or submit as an individual, as desired. You need to set up an Assignment 3 group even if it has the same members as your previous Assignment

group.

Marking

There will again be 5 marks for this assignment. Also, there is a (small) possibility for 1 bonus mark (see below).

- 1 mark for your file `test.log` which shows the log of your correct test execution on **a standard Linux undergraduate machine**, and your `readme.txt` file with team information.
- 1 mark for passing our public test cases
- 1 mark for code quality. More details on page: all assignments - rules and policies. Also consider the feedback from assignment 1 style marking for your assignment (if any).
- 2 marks for passing our private test cases

Code that does not run, or just hardcodes answers to public test cases, will receive a mark of 0. If your code needs fixes, you need to submit a revised version **before the submission deadline**. TA will not attempt to fix your code under any circumstances. Use the discussion forum or consult the teaching team in case of questions.

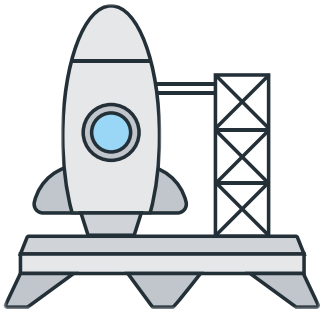
Up to one bonus mark **may** be awarded at the discretion of the instructor. This is expected to be rare - for an exceptional implementation, especially of the probabilistic policy, which is clearly faster than the other submissions, while still being correct. Of course any submission must follow our general assignment policies such as no "special tricks" with compiling etc. In my opinion, there is less scope for dramatic algorithmic improvements than in assignments 2 and 4, but - surprise us!

Last modified: Oct 15, 2026 by Martin Müller

Keep in mind, this submission will count for everyone in your Assignment 3 Groups group.

Choose a submission type

Upload | Canvas Studio | More



Drag a file here, or

Choose a file to upload

File permitted: TGZ

or

Canvas Files

Submit Assignment